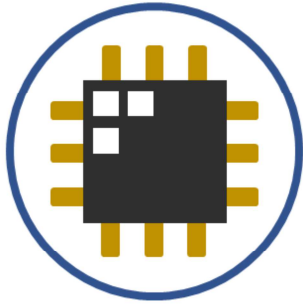
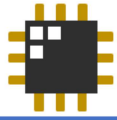




# EEE 143: Switching Theory and Digital Logic Design

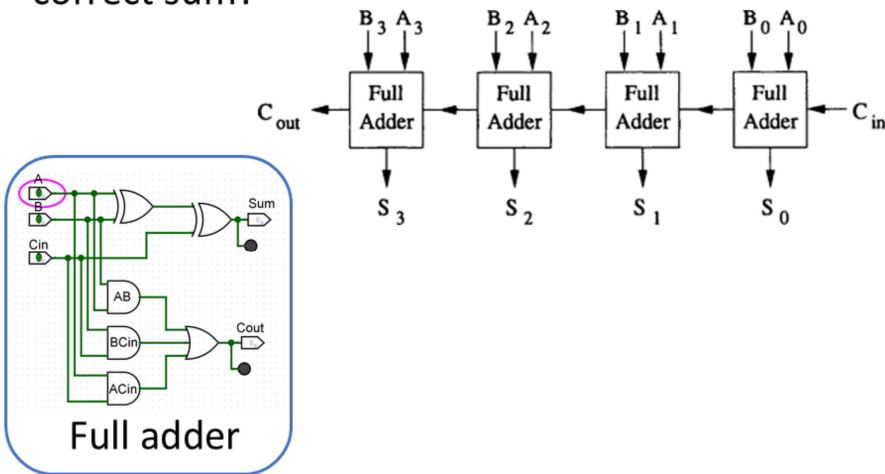
## Lecture 13: Sequential circuit non-idealities

1



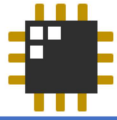
## Recap: Adder propagation delay

- Given a 4-bit ripple carry adder, what's the **longest path** to get the correct sum?



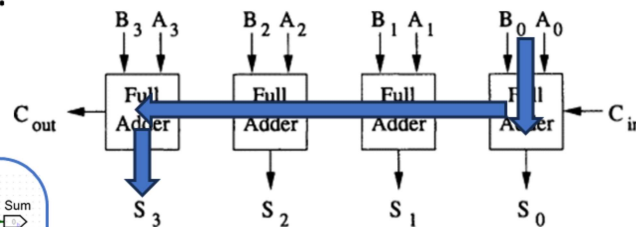
2

Let's recall that in a ripple carry adder, each full adder depends on the carry output of the previous one. This sequential dependency means the carry can't move forward until the previous stage finishes its calculation. As a result, the longest delay—what we call the critical path—happens when the carry has to propagate all the way from the least significant bit (LSB) to the most significant bit (MSB). This ripple effect through all stages is what gives the adder its name and is the main reason why its speed is limited by the number of bits.



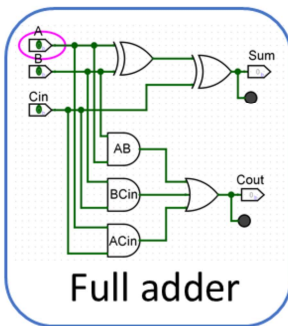
## Recap: Adder propagation delay

- Given a 4-bit ripple carry adder, what's the **longest path** to get the correct sum?



$$3 \times [(1 \text{ 2-input AND}) + (1 \text{ 3-input OR})] + (1 \text{ 2-input XOR})$$

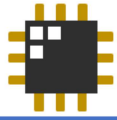
This is also called the critical path delay



Full adder

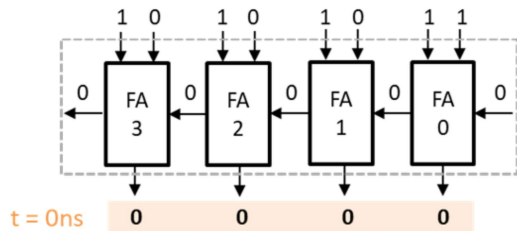
3

Let's also note how the actual logic gates inside the full adders contribute to the delay in a 4-bit ripple carry adder. Each full adder uses basic gates like AND, OR, and XOR, and each of these gates adds its own delay. From the slide, we saw that the delay can be broken down into three main stages for carry propagation—each stage includes a 2-input AND gate followed by a 3-input OR gate. Since the carry has to move through several of these stages across the adders, the delay accumulates as it travels from one bit to the next. On top of that, computing the final sum bit also takes time because it depends on a 2-input XOR gate. So, when we talk about the overall propagation delay, we're really referring to this entire chain of logic gates—the longest path a signal has to travel, which defines the critical path and ultimately limits how fast the ripple carry adder can operate.

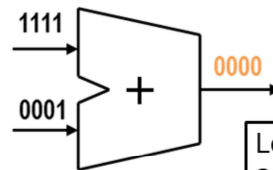


## Propagation delay on the output

- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)



Inputs changed but each FA takes some time to 'react'



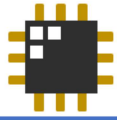
Let's say initially the adder was adding up **0000 + 0000** earlier. Due to delays, output does not change yet with the new inputs

\* For simplicity, assume sum and carry-out are output by each FA after 1ns from input change

4

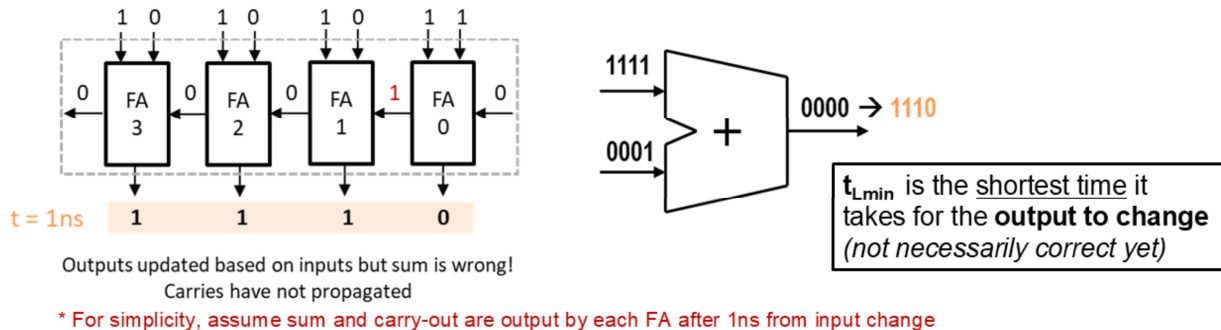
In this example, we are using the same circuit as before, but now we focus on understanding what to expect during operation. Imagine applying the inputs **1111 + 0001** simultaneously on the adder side. As we analyze the behavior over time, we can observe how the system processes these inputs at the nanosecond level. The key takeaway is to visualize the **glitches** that occur due to propagation delays. Initially, at **t = 0ns**, no changes have taken effect yet, so the output remains **0000**.





## Propagation delay on the output

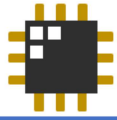
- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)



5

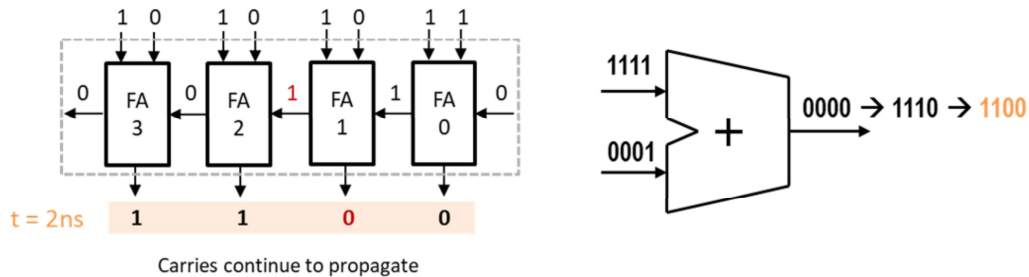
Let's also recall what happens during the initial moments of signal propagation in a 4-bit ripple carry adder. At time  $t = 1\text{ ns}$ , for example, the output of the first full adder (FA0) might already show a result of 0, with its carry-in ( $C_{in}$ ) equal to 1. However, it's crucial to understand that the subsequent full adders—FA1, FA2, and FA3—begin their operations simultaneously with FA0, even though their results are not yet valid. Due to this, the intermediate output at that moment might appear as 1110.

This leads us to the concept of  **$t_{Lmin}$** , which is the shortest time it takes for any output to begin changing—not necessarily producing the correct final result yet. It's a reflection of the fact that signals start to ripple through the circuit early, but the **correct** sum only becomes valid after the carry has fully propagated through all four stages. This again ties back to our discussion on propagation delay and how it defines when we can truly trust the output of a ripple carry adder.



## Propagation delay on the output

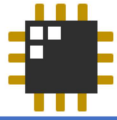
- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)



\* For simplicity, assume sum and carry-out are output by each FA after 1ns from input change

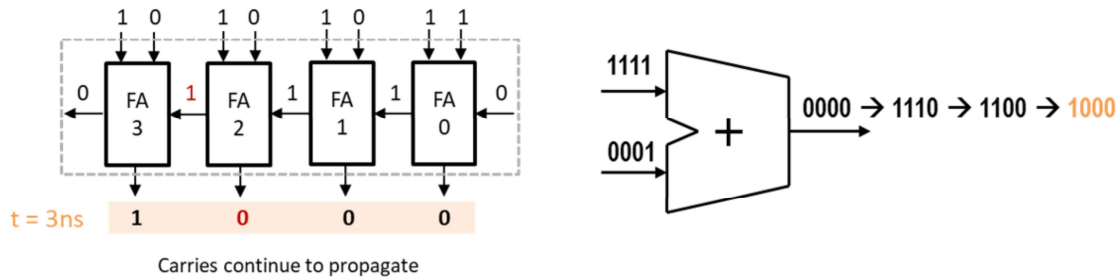
6

At  $t = 2ns$ , the carry output from **FA0** at  $t = 1ns$  is now available as an input to **FA1**. This means that **FA1** will perform a new addition incorporating this carry. As a result, the output of **FA1** updates to **0** and the carry-out of **FA1** is now **1**, reflecting the impact of the delayed carry propagation.



## Propagation delay on the output

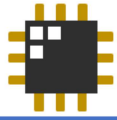
- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)



\* For simplicity, assume sum and carry-out are output by each FA after 1ns from input change

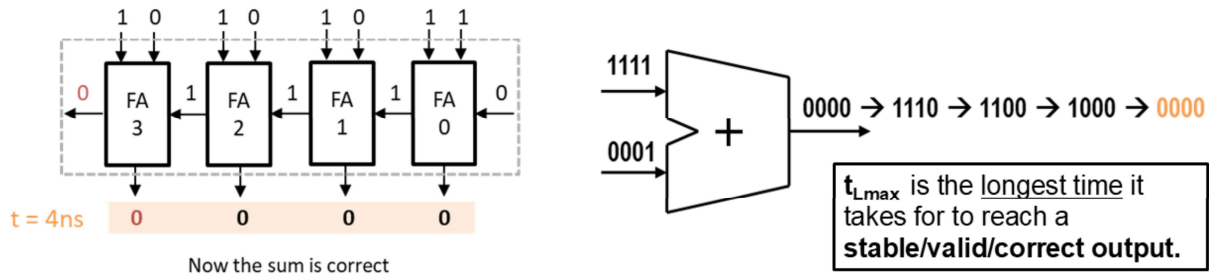
7

At **t = 3ns**, the carry output from **FA1** is now available and is added to the previous output of **FA2**. As a result, **FA2** updates its sum output to **0**, while its carry-out becomes **1**, continuing the propagation delay through the circuit.



## Propagation delay on the output

- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)

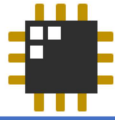


8

Now let's remember what happens as the carry signal continues to ripple through the stages of a 4-bit ripple carry adder. By the time we reach  $t = 4\text{ ns}$ , the carry output from FA2 has finally propagated to FA3 and been included in its computation. As a result, FA3's sum output becomes 0, completing the correct final result. At this point, the entire circuit has stabilized—this is when all the intermediate transitions have settled and the output is finally valid.

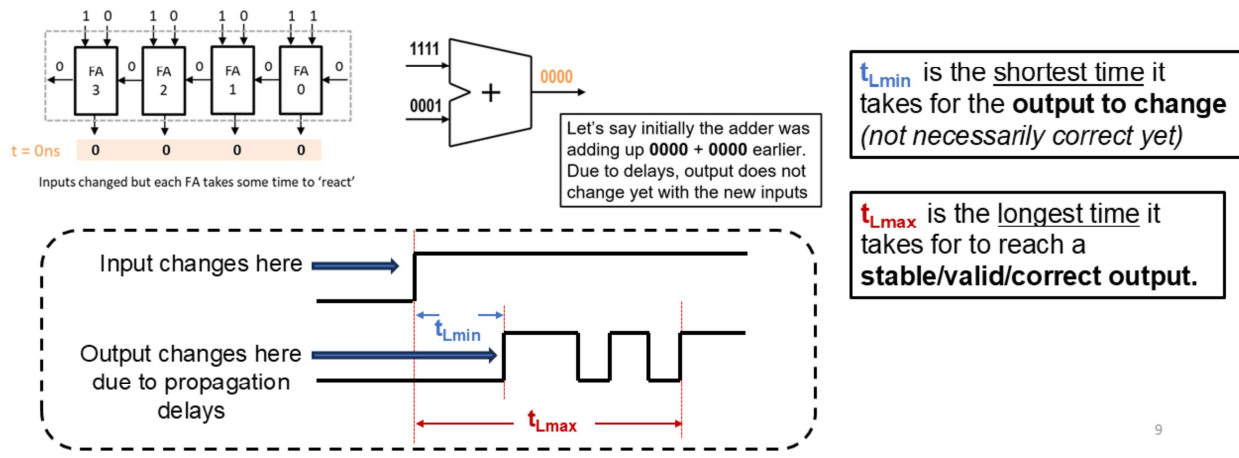
This moment represents  **$t_{L\max}$** , which is the longest time it takes for the output to become stable and correct. It marks the end of the propagation delay, after which the sum can be reliably read.

So in summary, while  **$t_{L\min}$**  indicates when the output first starts to respond,  **$t_{L\max}$**  tells us when the output is fully accurate and the addition is complete.

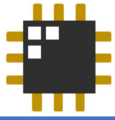


## Propagation delay on the output

- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)



These timings illustrate how input changes lead to delayed output changes due to propagation delay, as we've already seen in the previous slides. Again,  **$t_{Lmin}$**  is the shortest time after an input change when the output starts to respond, though it may still be incorrect.  **$t_{Lmax}$**  is the longest time it takes for the output to fully stabilize and become valid.



## Timing considerations for sequential circuits

- We often want a circuit to be as fast as possible. However, it takes some time for signals to propagate (propagation delay)
- We need to ensure that we are still operating correctly (we should be getting the **correct** values every clock cycle)

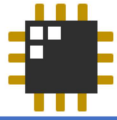
**Now, let's consider how *propagation delays* affect synchronous sequential circuit functionality**

10

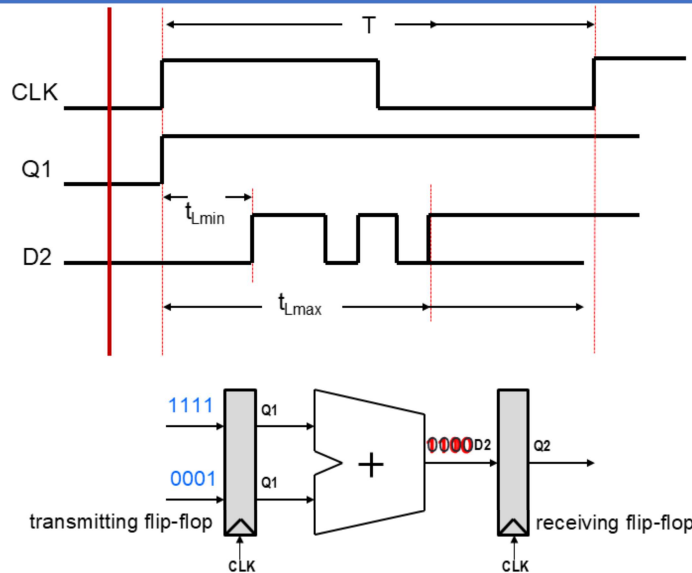
Now, when it comes to **timing considerations for sequential circuits**, things become even more critical. In digital design, we naturally want our circuits to operate as fast as possible to improve performance. However, we have to account for the fact that **signals don't propagate instantaneously**—they take time to travel through logic gates and flip-flops, which is what we refer to as **propagation delay**.

In **synchronous sequential circuits**, where operations are driven by a clock, this delay directly affects functionality. If the clock cycles too quickly—before all signals have settled—we risk capturing incorrect values. That's why **correct timing** is not just about speed but also about **reliability**. We need to ensure that every clock cycle gives us valid and stable outputs.

So, as we now consider how propagation delays impact these types of circuits, we'll focus on ensuring that the system stays **functionally correct**—that is, all computations complete before the next clock edge, so the circuit behaves as expected.



## Timing considerations for sequential circuits



### Assuming ideal flip-flops:

You need to wait for the worst-case operation (**longest delay,  $t_{Lmax}$** ) to finish before the receiving flip-flop captures it!

$$T \geq t_{Lmax}$$

clock period  $\rightarrow$   $T$   $\leftarrow$  longest delay  
should be greater than or equal to

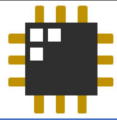
11

Now, let's talk about timing under the assumption of **ideal flip-flops**—that is, flip-flops with no internal delay of their own. Even with ideal flip-flops, the rest of the circuit still experiences **propagation delay**. That's why we need to consider the **worst-case scenario**, where the data takes the longest possible time to travel through the combinational logic. This is represented by  **$t_{Lmax}$** .

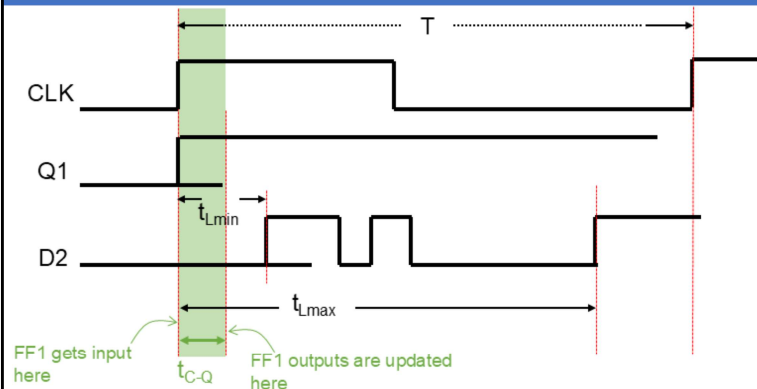
To ensure correct operation, the system clock period  **$T$**  must be **at least equal to  $t_{Lmax}$** . In other words,

$$T \geq t_{Lmax}$$

This gives the circuit enough time for the data to propagate and settle before the receiving flip-flop samples it on the next clock edge. If  **$T$**  is shorter than  **$t_{Lmax}$** , the flip-flop may latch in incorrect or incomplete data, leading to functional errors. So, while we want high-speed operation, the clock must still respect the longest path delay in the logic.



## Flipflop propagation time



But flip-flops are not ideal.

### Flipflop clock-to-Q delay ( $t_{C-Q}$ ):

- The time it takes to update the flipflop output *after the active edge of the clock*
- Essentially, the propagation delay of the flipflop

It takes some time after the clock edge before the output of the transmitting flip-flop is updated with its new inputs.

Only after the output of the transmitting flip-flop is updated can we start the 'actual' operation, and hence start 'counting'  $t_{Lmin}$  and  $t_{Lmax}$ .

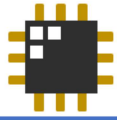
12

Now let's take it a step further—**flip-flops are not ideal** in real-world circuits. One key non-ideal behavior is the **clock-to-Q delay**, denoted as  **$t_{C-Q}$** . This is the amount of time it takes for the output of a flip-flop to update after the active edge of the clock signal. In simple terms, it's the **flip-flop's own internal propagation delay**.

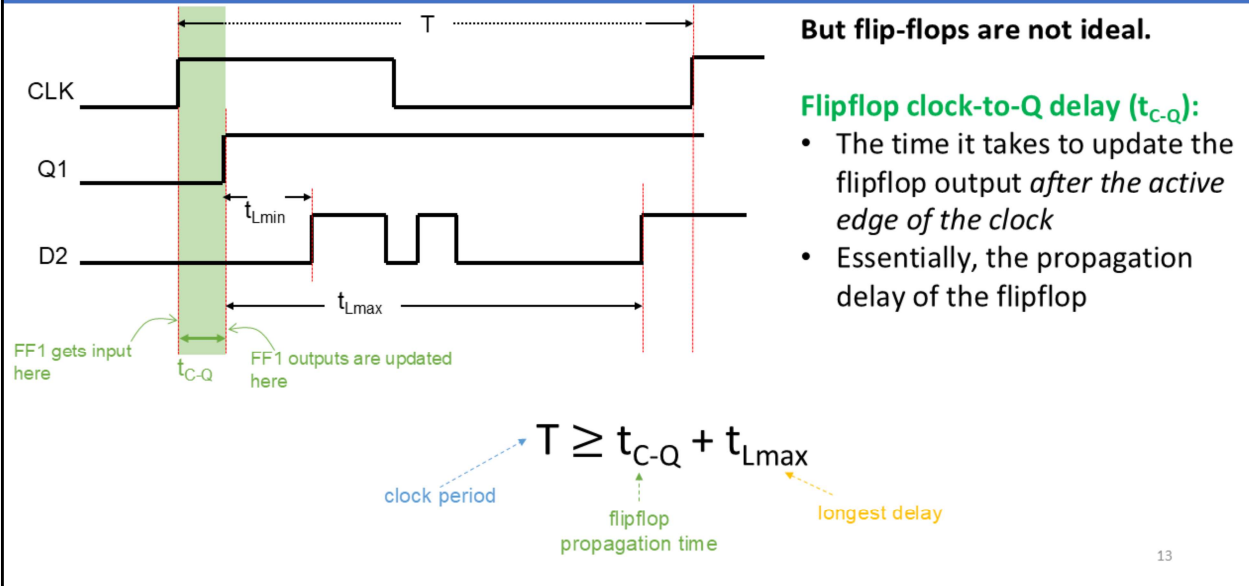
So what does this mean for timing? After the clock edge, the transmitting flip-flop doesn't update its output instantly—it takes  **$t_{C-Q}$**  before its new value appears at the output. Only *after* this happens can the combinational logic begin its actual operation. That means the timing of the whole data path is offset by this delay. From that point onward, we start counting the familiar  **$t_{Lmin}$**  and  **$t_{Lmax}$**  to track how long it takes for the output to respond and eventually stabilize.

This makes  **$t_{C-Q}$**  an important part of the total timing equation, because it adds an extra delay before the receiving flip-flop can even begin to process new data. So, in reality, our system clock period  **$T$**  must be large enough not only to cover  **$t_{Lmax}$** , but also to include the  **$t_{C-Q}$**  delay from the transmitting flip-flop.





## Flipflop propagation time

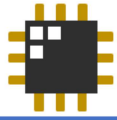


Building on what we've just discussed, now that we know flip-flops are not ideal and have a **clock-to-Q delay ( $t_{C-Q}$ )**, we need to adjust our timing equation accordingly. Previously, we said that the clock period  $T$  must be at least equal to  $t_{Lmax}$ , to allow the data to fully propagate through the combinational logic. But now we know that the data doesn't even *start* moving through the logic until  $t_{C-Q}$  time has passed—this is the time it takes for the transmitting flip-flop to update its output after the clock edge.

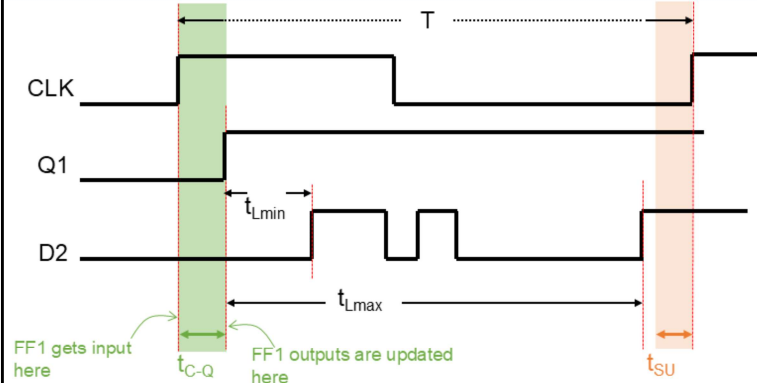
So, to ensure that the receiving flip-flop captures *correct* and *stable* data, the clock period  $T$  must account for **both** the delay in the flip-flop itself **and** the longest propagation delay through the logic. This leads us to the refined timing requirement:

$$T \geq t_{C-Q} + t_{Lmax}$$

This equation ensures the system has enough time for the data to leave the transmitting flip-flop *after* the clock edge, fully propagate through the logic path, and arrive at the receiving flip-flop *before* the next clock edge. This is a key part of designing reliable synchronous systems.



## Flipflop setup time



But flip-flops are not ideal.

### Flipflop setup time ( $t_{SU}$ ):

- Minimum time needed for the input of the flipflop to become stable before the active edge of the clock

The *correct* input to the receiving flip-flop should be held stable for some time ( $t_{SU}$ ) before it is read at the clock edge.

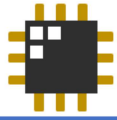
Because the input should be *correct*, we can only start 'counting'  $t_{SU}$  after  $t_{Lmax}$ .<sup>14</sup>

Now, let's talk about **flip-flop setup time ( $t_{SU}$ )**, which is another crucial aspect of timing in sequential circuits.  **$t_{SU}$**  refers to the **minimum time** required for the input of the flip-flop to become stable **before** the active edge of the clock. In other words, the input signal to the receiving flip-flop needs to remain steady and stable for a certain period,  **$t_{SU}$** , before the clock triggers the flip-flop to latch the value.

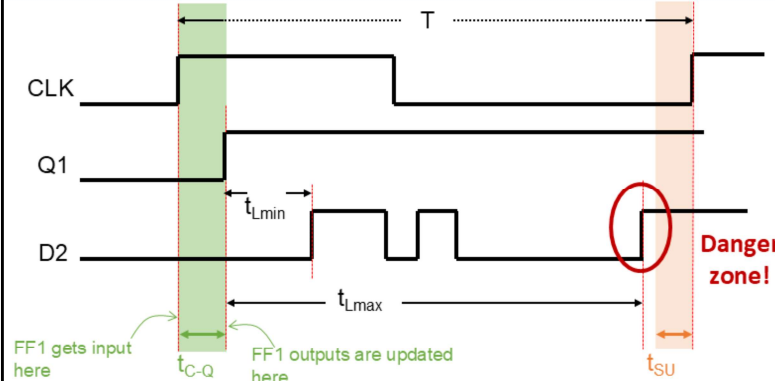
This is important because, for the receiving flip-flop to correctly capture the input, the signal must be stable. If the input changes too close to the clock edge, the flip-flop may capture an incorrect value, or the value might not be fully stable yet, leading to errors. The flip-flop needs enough time to "settle" the input before the clock edge.

Now, just like with  **$t_{C-Q}$**  and  **$t_{Lmax}$** ,  **$t_{SU}$**  affects the overall timing of the system. Since we want the input to be stable before the clock edge, we can't start counting  **$t_{SU}$**  until  **$t_{Lmax}$**  has passed. In other words, the signal propagation needs to be fully completed through the combinational logic (which takes time equal to  **$t_{Lmax}$** ) before we can ensure the input to the receiving flip-flop is stable enough for it to capture correctly.

So, the relationship becomes: the receiving flip-flop can only correctly latch the input if it has been stable for **t<sub>SU</sub>** after **t<sub>Lmax</sub>**, and before the next clock edge. This setup time must be respected to maintain the correct functionality of the circuit.



## Flipflop setup time



But flip-flops are not ideal.

### Flipflop setup time ( $t_{SU}$ ):

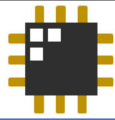
- Minimum time needed for the input of the flipflop to become stable before the active edge of the clock

If the input to the flipflop still changed *within the setup time window*, the flipflop output becomes unpredictable (we don't want this to happen)

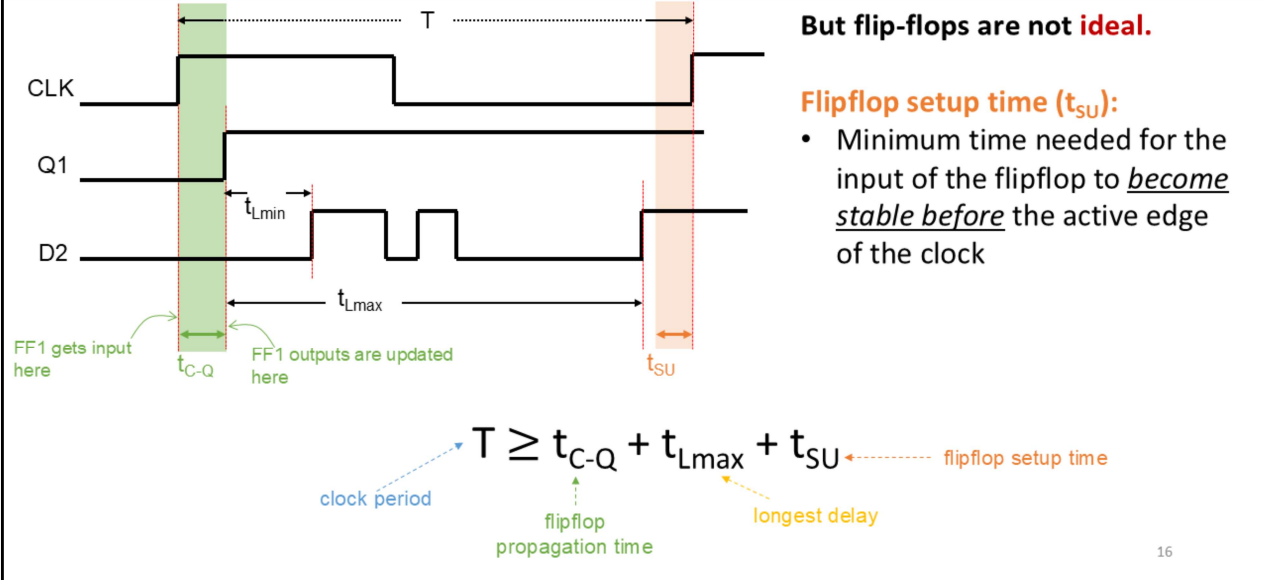
15

To build on what we've discussed regarding **setup time ( $t_{SU}$ )**, it's critical to understand what happens if the input to the flip-flop doesn't stay stable for the required  **$t_{SU}$**  before the clock edge. If the input signal still changes **within the setup time window**—meaning it is not stable for the required duration before the clock triggers—the flip-flop will not be able to correctly latch the input.

This results in an **unpredictable output** from the flip-flop. Essentially, the flip-flop may capture an incorrect or incomplete value, or it might even fail to latch any value at all. This is a situation we absolutely want to avoid, because it introduces **timing errors** into the circuit, leading to instability or malfunction in the system.



## Flipflop setup time



Now, the equation for the clock period,  $T$ , needs to account for several important delays to ensure the correct operation of the system. We've already considered the propagation delays and flip-flop behaviors. Now, combining everything, we can express the total required clock period as:

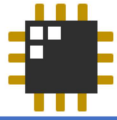
$$T \geq t_{C-Q} + t_{Lmax} + t_{SU}$$

Here's why each term is important:

- **$t_{C-Q}$ :** The **clock-to-Q delay** is the time it takes for the transmitting flip-flop to update its output after the clock edge. This delay must be accounted for because it affects when the output becomes available for the next stage of the circuit.
- **$t_{Lmax}$ :** The **longest propagation delay** through the combinational logic ensures that all signals are fully propagated and settled before they reach the next flip-flop.
- **$t_{SU}$ :** The **setup time** ensures that the input to the receiving flip-flop remains stable for enough time before the next clock edge, allowing the flip-flop to reliably latch the correct value.

Together, these delays must be covered within the clock period  $T$ . The clock

period has to be long enough to allow for the data to propagate through all stages, be updated by the flip-flops, and be held stable for the setup time—before the next clock edge arrives. If the clock period is too short, any one of these delays could cause timing errors, leading to unpredictable or incorrect outputs, which is why this equation is critical for the proper functioning of synchronous sequential circuits.



## Timing considerations for sequential circuits

System > About

Galileo  
CHRONOS

① Device specifications

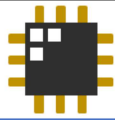
|             |                                                   |
|-------------|---------------------------------------------------|
| Device name | Galileo                                           |
| Processor   | Intel(R) Core(TM) i9-9900K CPU @ 3.60GHz 3.60 GHz |

This is the clock frequency of the computer processor where this presentation was made...

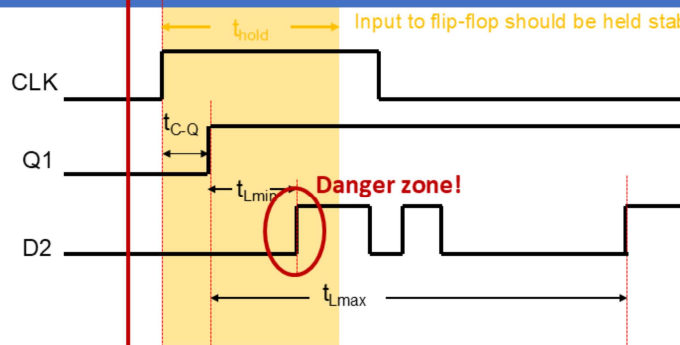
$$T \geq t_{C-Q} + t_{Lmax} + t_{SU}$$

These considerations set a limit to how short the clock period is, and hence how fast the frequency ( $f = 1/T$ ) of the clock can be.

The considerations we've discussed, like  **$t_{C-Q}$** ,  **$t_{Lmax}$** , and  **$t_{SU}$** , set a limit on how short the clock period  **$T$**  can be, which in turn limits the maximum clock frequency ( **$f = 1/T$** ) of the system. The screenshot showing the laptop processor's clock frequency reflects this—while a higher frequency means faster operations, it's constrained by these propagation delays. The clock period cannot be too short, or else errors could occur, ensuring the system operates within these timing limits for correct functionality.



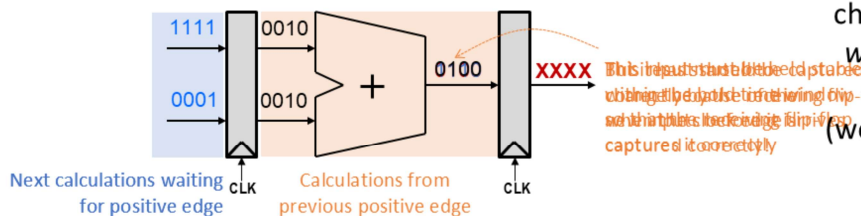
## Flipflop hold time



### Flipflop hold time ( $t_H$ ):

- Minimum time needed for the input of the flipflop to remain stable after the active edge of the clock

If the input to the flipflop changed *within the hold time window*, the flipflop output becomes unpredictable (we don't want this to happen)



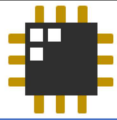
18

There is another aspect that we need to check: hold time. This is a different consideration from when we looked at setup time earlier.

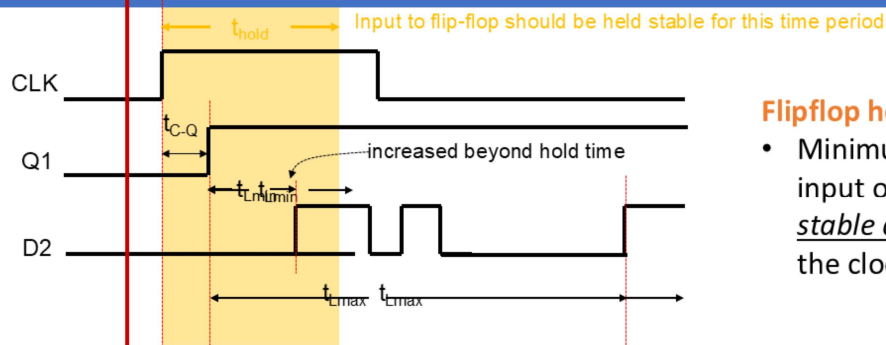
Now, our problem is that once a valid answer is available at the inputs of the receiving flip-flops, we still have to hold that input *after* the edge of the clock for the receiving flip-flop to effectively capture it. Imagine taking a photo with a camera (at the edge of the clock), you want to be ready some time *before* you take the photo and hold still *after* you take the photo so the result isn't blurred.

But if the next clock edge has come, that means there is also new inputs at the transmitting flip-flop. If our adder is so fast that the output starts to change while we are still not done "capturing" the previous answer, the flip-flop might store the wrong answer!



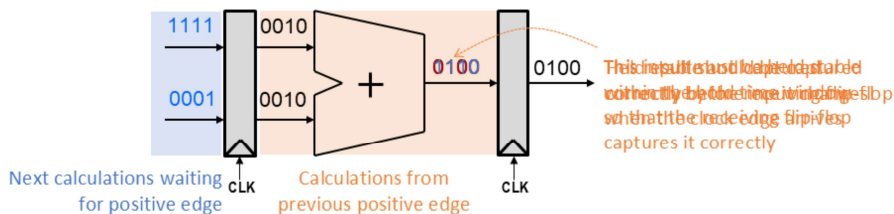


## Flipflop hold time



### Flipflop hold time ( $t_H$ ):

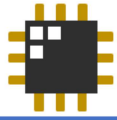
- Minimum time needed for the input of the flipflop to remain stable after the active edge of the clock



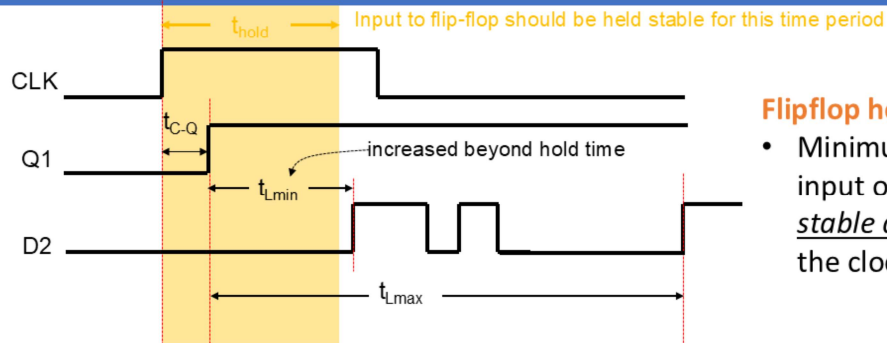
19

To avoid this, the input must remain stable during the **hold time window**—the period after the clock edge but before the input can safely change.

The key idea here is that the input must be held stable and **captured correctly** by the receiving flip-flop before any further changes occur to the input signal. If this condition isn't met, the output could become unpredictable, just like with setup time violations. Therefore,  $t_H$  is just as crucial as  $t_{SU}$  in ensuring reliable flip-flop operation and correct data capture in synchronous circuits.



## Flipflop hold time



### Flipflop hold time ( $t_H$ ):

- Minimum time needed for the input of the flipflop to remain stable after the active edge of the clock

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

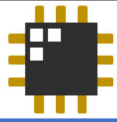
flipflop hold time

flipflop propagation time

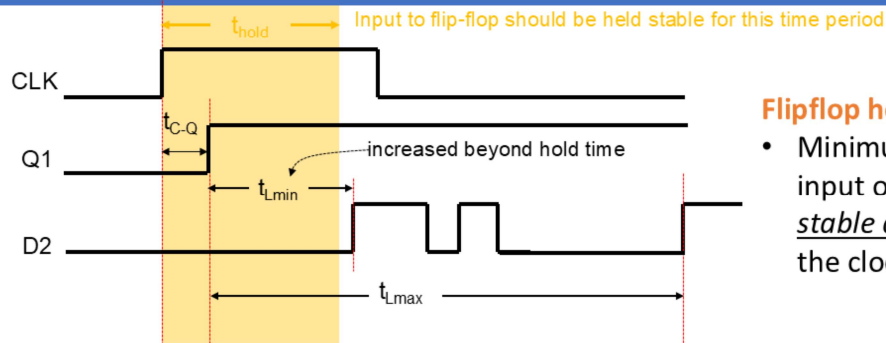
shortest logic delay

20

What we can do is increase the  $t_{Lmin}$  to satisfy the condition in the slide.



## Flipflop hold time



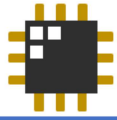
### Flipflop hold time ( $t_H$ ):

- Minimum time needed for the input of the flipflop to remain stable after the active edge of the clock

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

Note that this inequality is independent of the clock period!  
This is another timing constraint to satisfy to guarantee correct operation

This inequality involving **hold time (tH)** is **independent of the clock period**. Unlike setup time, which is affected by the clock period, hold time must be respected regardless of the clock's speed. It's a separate timing constraint that ensures the input to the flip-flop remains stable long enough after the clock edge to guarantee correct operation. If the input changes too soon, even with an ideal clock period, the flip-flop could capture an incorrect value. Thus, satisfying **tH** is crucial for reliable operation, regardless of the clock's timing.



## Timing considerations for sequential circuits

$$T \geq t_{C-Q} + t_{Lmax} + t_{SU}$$

clock period  $T$  (blue dashed arrow), flipflop propagation time  $t_{C-Q}$  (green dashed arrow), longest logic delay  $t_{Lmax}$  (yellow dashed arrow), flipflop setup time  $t_{SU}$  (orange dashed arrow)

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

flipflop hold time  $t_{hold}$  (orange dashed arrow), flipflop propagation time  $t_{C-Q}$  (green dashed arrow), shortest logic delay  $t_{Lmin}$  (red dashed arrow)

**Both of these (independent) inequalities must be satisfied to guarantee correct operation!**

22

To guarantee correct operation in a synchronous circuit, **both** of these timing inequalities must be satisfied:

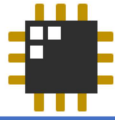
**1.  $T \geq t_{C-Q} + t_{Lmax} + t_{SU}$**

This ensures the clock period  $T$  is long enough to account for the clock-to-Q delay, the longest propagation delay through the logic, and the setup time needed for the input to the receiving flip-flop to become stable before the clock edge.

**2.  $t_{Hold} \leq t_{C-Q} + t_{Lmin}$**

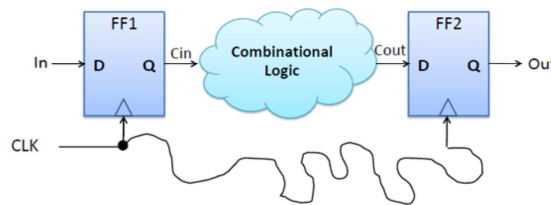
This inequality ensures that the input to the flip-flop remains stable for the required hold time  $t_{Hold}$  after the clock edge, preventing the flip-flop from capturing an incorrect value.

Both conditions are essential because they guarantee that data is correctly propagated and captured by the flip-flops, maintaining the integrity and stability of the circuit's operation. If either of these inequalities is violated, the circuit may produce incorrect or unpredictable results.



## Other clock non-idealities

- There are other clock non-idealities (but are out of EEE 143 scope)
  - **Clock skew** (clock signal arrives at different times for different flipflops)



Results in a **clock phase shift** for different flipflops

Mainly due to wire delays

Generally, **we try to route the clock signal to the flipflops with same length wires** so that the effective clock skew between all flipflops is 0 (or near 0)

Clock skew effect is *very* significant in the GHz operating clock frequencies!

23

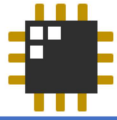
While we've covered several key timing considerations, there are other **clock non-idealities** that can affect circuit performance, though they are outside the scope of this course. One such non-ideality is **clock skew**.

**Clock skew** occurs when the clock signal reaches different flip-flops at slightly different times, resulting in a **clock phase shift** between flip-flops. This is typically caused by **wire delays**—the time it takes for the clock signal to propagate through the circuit's wiring.

To minimize the effect of clock skew, we generally try to route the clock signal to all flip-flops using wires of approximately the same length, ensuring that the clock signal arrives at each flip-flop at the same time, or as close to the same time as possible. The goal is to have the **effective clock skew** between flip-flops be **zero** or at least **nearly zero**.

Clock skew becomes particularly **significant** when operating at **GHz clock frequencies**. At these high speeds, even small differences in clock arrival times can have a noticeable impact on the performance and reliability of the circuit. Thus, clock skew needs to be carefully managed in high-speed designs

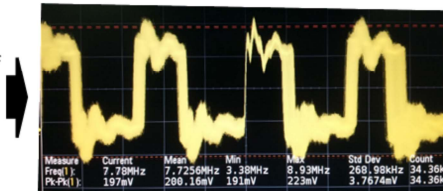
to avoid timing errors and maintain system stability.



## Other clock non-idealities

- There are other clock non-idealities (but are out of EEE 143 scope)
  - **Clock jitter** (clock edges may shift cycle-per-cycle)

Oscilloscope capture of an on-chip clock signal illustrating clock jitter



Oscillator non-idealities of the SmartWire chip. Captured in EEEl Rm. 420 (A. Chua, 2016)

Results in a clock period variation seen by different flipflops

Mainly due to circuit noise and supply voltage variations

Generally, we try to minimize this through proper circuit design and routing techniques to reduce noise

In practice, we just account for the worst-case clock jitter and include that in the inequalities earlier to consider the worst-case scenario

24

Another important clock non-ideality is **clock jitter**, which refers to the **variation** in the timing of clock edges from one cycle to the next. This means that the clock signal is not perfectly consistent, and the edges of the clock may shift slightly from cycle to cycle. The impact of clock jitter is illustrated through an **oscilloscope capture** of an on-chip clock signal, which would show this variation in clock edges.

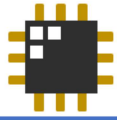
Clock jitter leads to **variation in the clock period** as seen by different flip-flops in the circuit. The inconsistency can cause timing issues because each flip-flop may receive clock signals with slightly different timing, disrupting synchronization.

The main causes of clock jitter are **circuit noise** and **supply voltage variations**, which introduce fluctuations in the clock signal. To minimize jitter, circuit designers use proper **circuit design** and **routing techniques** to reduce noise and ensure the clock signal remains as stable as possible.

In practice, designers often **account for worst-case clock jitter** by including it in the timing inequalities we've discussed earlier. This ensures the system is

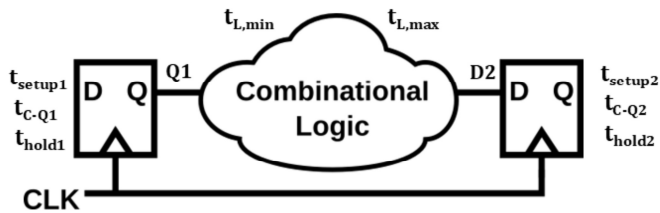
designed to tolerate the maximum possible jitter, providing robustness against this non-ideality and ensuring the circuit continues to function correctly even in the worst-case scenario.





## Practice #1

- Given the circuit below and the following timing parameters

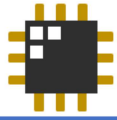


**Determine the minimum clock period  $T$  and the required  $T_{Lmin}$  to guarantee correct operation.**

|                                        |            |
|----------------------------------------|------------|
| $t_{Logic}$<br>(for correct output)    | 10 – 40 ns |
| $t_{Lmin}$<br>(first change in output) | X          |
| $t_{setup1}$                           | 3 ns       |
| $t_{C-Q1}$                             | 4 ns       |
| $t_{hold1}$                            | 5 ns       |
| $t_{setup2}$                           | 6 ns       |
| $t_{C-Q2}$                             | 7 ns       |
| $t_{hold2}$                            | 8 ns       |

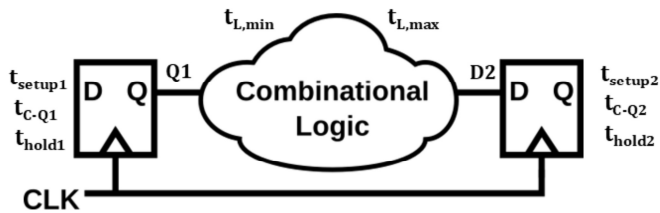
25

Let's begin with the first example: we need to determine the **minimum clock period ( $T$ )** and the required  **$T_{Lmin}$**  to guarantee correct operation.



## Practice #1 Part 1

- Given the circuit below and the following timing parameters



Minimum clock period:

$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

|                                        |            |
|----------------------------------------|------------|
| $t_{Logic}$<br>(for correct output)    | 10 – 40 ns |
| $t_{Lmin}$<br>(first change in output) | X          |
| $t_{setup1}$                           | 3 ns       |
| $t_{C-Q1}$                             | 4 ns       |
| $t_{hold1}$                            | 5 ns       |
| $t_{setup2}$                           | 6 ns       |
| $t_{C-Q2}$                             | 7 ns       |
| $t_{hold2}$                            | 8 ns       |

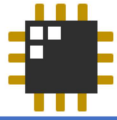
What values of  $t_{C-Q}$  and  $t_{SU}$  should we follow?

26

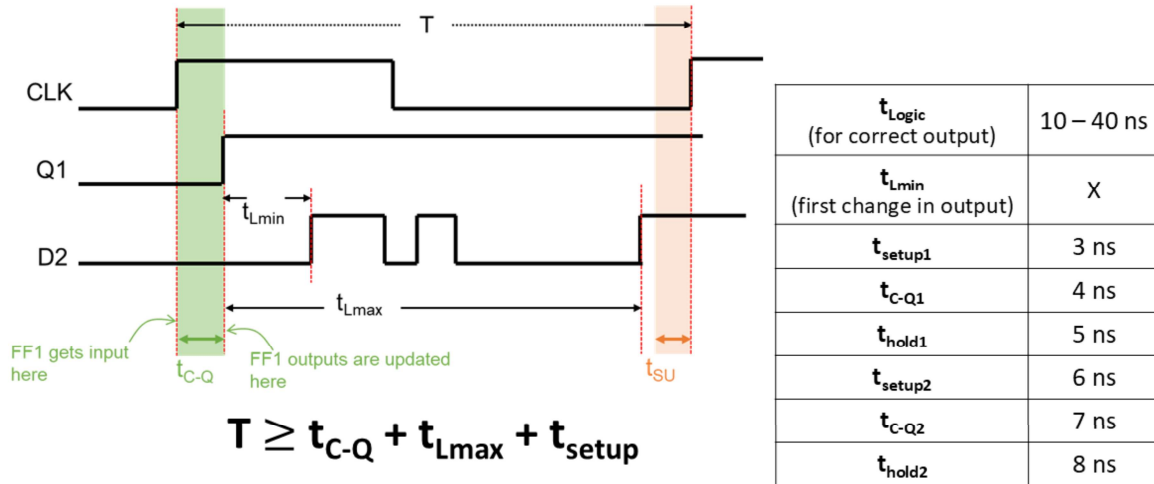
Note that the equation for the **minimum clock period** is given by:

$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

Now, what values of  $t_{C-Q}$  and  $t_{SU}$  should we follow?



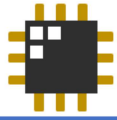
## Practice #1 Part 1



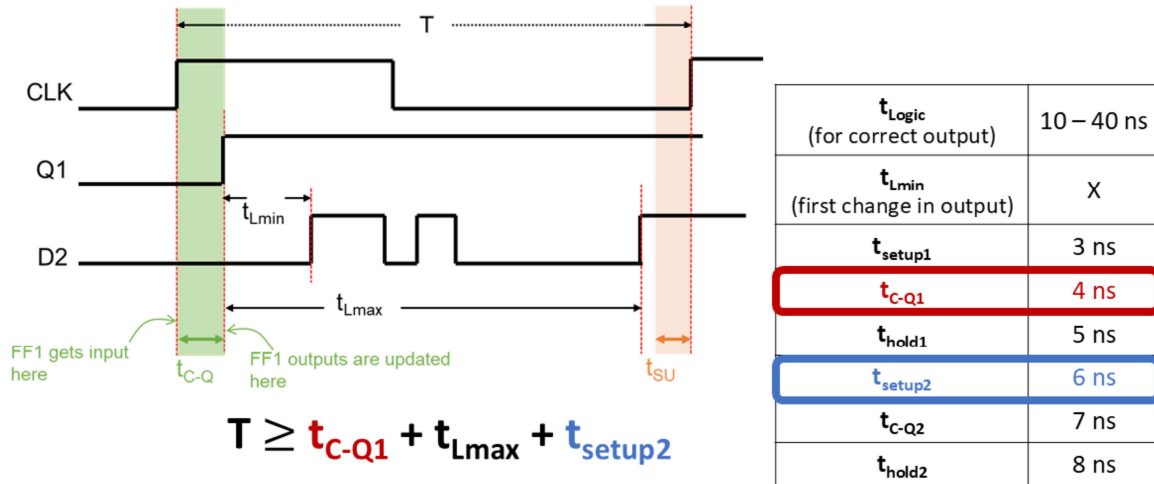
What values of  $t_{C-Q}$  and  $t_{SU}$  should we follow?

27

To determine the minimum clock period for two connected flip-flops, consider the maximum value of  $t_{Logic}$   $t_{C-Q}$  of the first flip-flop and the  $t_{setup}$  of the second flip-flop.



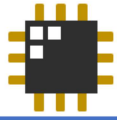
## Practice #1 Part 1



Always imagine the data flow of the circuit!

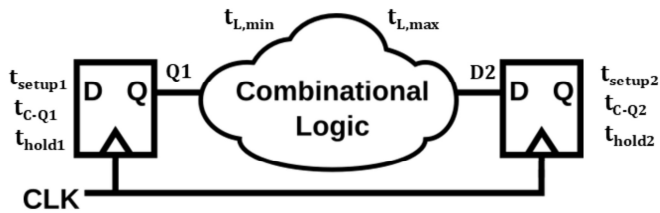
28

Here,  $t_{C-Q1} = 4 \text{ ns}$ ,  $t_{Lmax} = 40 \text{ ns}$ , and  $t_{setup2} = 6 \text{ ns}$ .



## Practice #1 Part 1

- Given the circuit below and the following timing parameters



Minimum clock period:

$$T \geq 4\text{ns} + 40\text{ns} + 6\text{ns}$$

$$T \geq 50\text{ns}$$

|                                               |            |
|-----------------------------------------------|------------|
| $t_{\text{Logic}}$<br>(for correct output)    | 10 – 40 ns |
| $t_{\text{Lmin}}$<br>(first change in output) | X          |
| $t_{\text{setup1}}$                           | 3 ns       |
| $t_{\text{c-Q1}}$                             | 4 ns       |
| $t_{\text{hold1}}$                            | 5 ns       |
| $t_{\text{setup2}}$                           | 6 ns       |
| $t_{\text{c-Q2}}$                             | 7 ns       |
| $t_{\text{hold2}}$                            | 8 ns       |

29

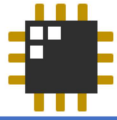
Applying the values to our equation:

$$T \geq t_{\text{c-Q1}} + t_{\text{Lmax}} + t_{\text{setup2}}$$

$$T \geq 4\text{ns} + 40\text{ns} + 6\text{ns}$$

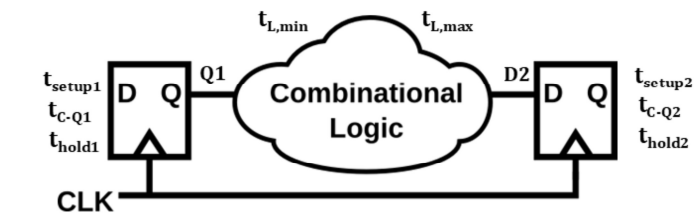
$$T \geq 50\text{ns}$$

So, the **minimum clock period T** must be **at least 50ns** to ensure the circuit operates correctly under the worst-case timing conditions.



## Practice #1 Part 2

- Given the circuit below and the following timing parameters



$t_{Lmin}$  calculation:

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

|                                        |            |
|----------------------------------------|------------|
| $t_{Logic}$<br>(for correct output)    | 10 – 40 ns |
| $t_{Lmin}$<br>(first change in output) | X          |
| $t_{setup1}$                           | 3 ns       |
| $t_{C-Q1}$                             | 4 ns       |
| $t_{hold1}$                            | 5 ns       |
| $t_{setup2}$                           | 6 ns       |
| $t_{C-Q2}$                             | 7 ns       |
| $t_{hold2}$                            | 8 ns       |

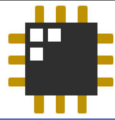
**What values of  $t_{C-Q}$  and  $t_{hold}$  should we follow?**

30

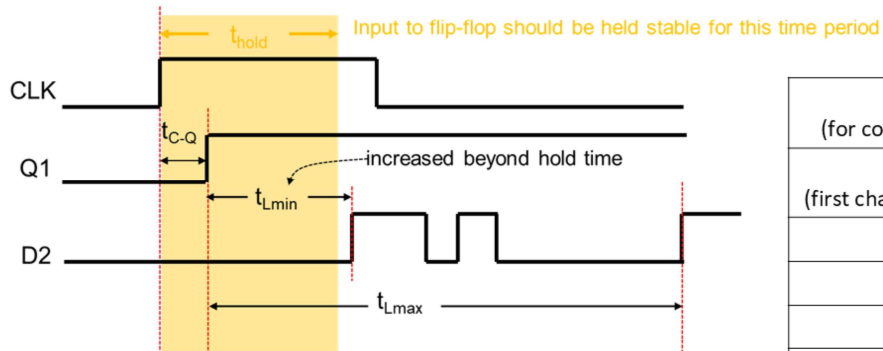
Now, moving on to the calculation of  $t_{Lmin}$ , we use the inequality:

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

This ensures that the data remains stable long enough after the clock edge for the receiving flip-flop to capture it correctly.



## Practice #1 Part 2



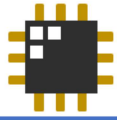
$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

|                                        |            |
|----------------------------------------|------------|
| $t_{Logic}$<br>(for correct output)    | 10 – 40 ns |
| $t_{Lmin}$<br>(first change in output) | X          |
| $t_{setup1}$                           | 3 ns       |
| $t_{C-Q1}$                             | 4 ns       |
| $t_{hold1}$                            | 5 ns       |
| $t_{setup2}$                           | 6 ns       |
| $t_{C-Q2}$                             | 7 ns       |
| $t_{hold2}$                            | 8 ns       |

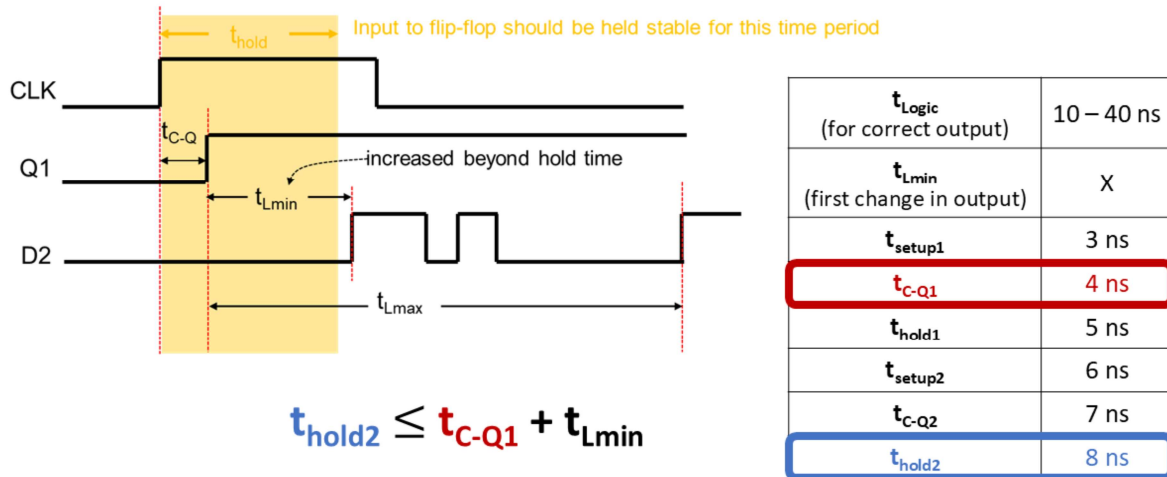
**What values of  $t_{C-Q}$  and  $t_{hold}$  should we follow?**

31

To determine the  $t_{Lmin}$  for two connected flip-flops, consider the  $t_{C-Q}$  of the first flip-flop and the  $t_{hold}$  of the second flip-flop.



## Practice #1 Part 2

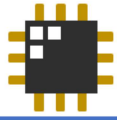


Always imagine the data flow of the circuit!

32

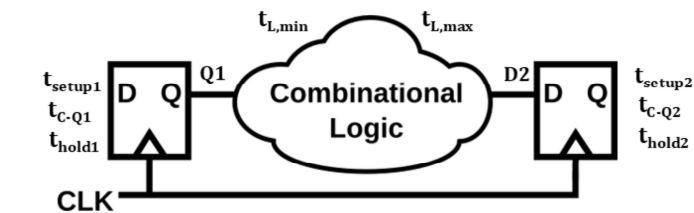
Here, we'll use  $t_{C-Q1} = 4$  ns and  $t_{hold2} = 8$  ns.





## Practice #1 Part 2

- Given the circuit below and the following timing parameters



$t_{Lmin}$  calculation:

$$8ns \leq 4ns + t_{Lmin}$$

$$t_{Lmin} \geq 4ns$$

|                                        |            |
|----------------------------------------|------------|
| $t_{Logic}$<br>(for correct output)    | 10 – 40 ns |
| $t_{Lmin}$<br>(first change in output) | <u>4ns</u> |
| $t_{setup1}$                           | 3 ns       |
| $t_{c-Q1}$                             | 4 ns       |
| $t_{hold1}$                            | 5 ns       |
| $t_{setup2}$                           | 6 ns       |
| $t_{c-Q2}$                             | 7 ns       |
| $t_{hold2}$                            | 8 ns       |

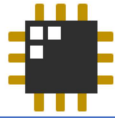
33

Using the inequality:

$$8ns \leq 4ns + t_{Lmin}$$

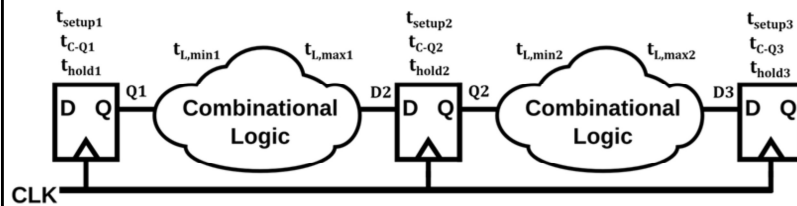
$$t_{Lmin} \geq 4ns$$

So, the **minimum propagation delay ( $t_{Lmin}$ )** is **at least 4ns** to satisfy the hold time requirement.



## Practice #2

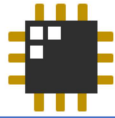
- Given the circuit below and the following timing parameters



**Determine the minimum clock period  $T$  and the required  $T_{Lmin}$  to guarantee correct operation.**

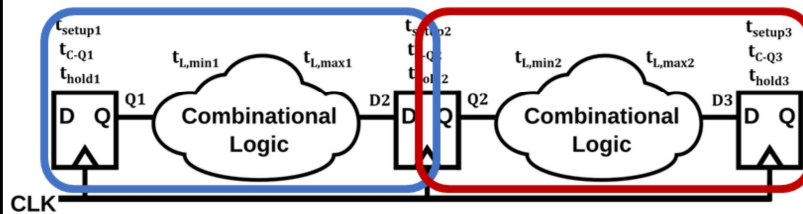
|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

Now, let's move on to the next example, where we will again solve for the **clock period  $T$**  and  **$T_{Lmin}$** .



## Practice #2

- Given the circuit below and the following timing parameters



There are **two paths** that we need to consider.

Each path is bounded by flipflops:

- A transmitting flipflop and a receiving flipflop

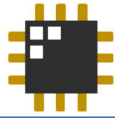
**Each of these paths must be considered independently!**

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{c-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{c-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{c-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

In this example, we need to consider **two separate paths**, each bounded by flip-flops—specifically, a **transmitting flip-flop** and a **receiving flip-flop**.

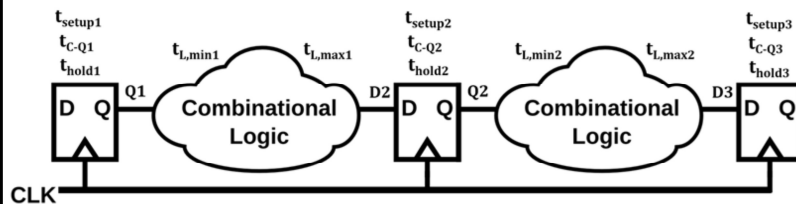
Each path has its own timing characteristics, as the signals traveling through each path will experience different delays, even though both paths involve flip-flops. These paths must be analyzed **independently**, because the timing constraints (like setup time, hold time, and propagation delay) may differ between them.

For each path, we must ensure that the data can propagate through the circuit correctly and be captured by the receiving flip-flop within the timing constraints. This independent analysis helps us guarantee that the circuit works reliably regardless of the specific path the signal takes.



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



Minimum clock period:

$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

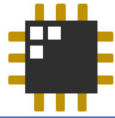
What values of  $t_{C-Q}$  and  $t_{SU}$  should we follow?

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

Note that the equation for the **minimum clock period** is given by:

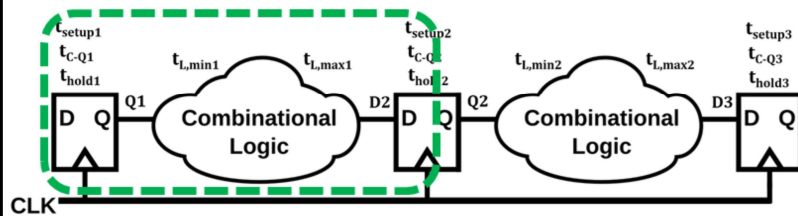
$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

Now, what values of  $t_{C-Q}$  and  $t_{SU}$  should we follow?



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



Path 1: Flipflop 1 to flipflop 2

$$T \geq t_{C-Q1} + t_{Lmax1} + t_{setup2}$$

$$T \geq 4ns + 40ns + 6ns$$

$$T \geq 50ns$$

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

Again, to determine the minimum clock period for two connected flip-flops, consider the maximum value of  $t_{Logic}$   $t_{C-Q}$  of the first flip-flop and the  $t_{setup}$  of the second flip-flop.

Here,  $t_{C-Q1} = 4 \text{ ns}$ ,  $t_{Lmax1} = 40 \text{ ns}$ , and  $t_{setup2} = 6 \text{ ns}$ .

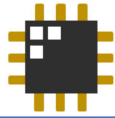
Applying the values to our equation:

$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

$$T \geq 4ns + 40ns + 6ns$$

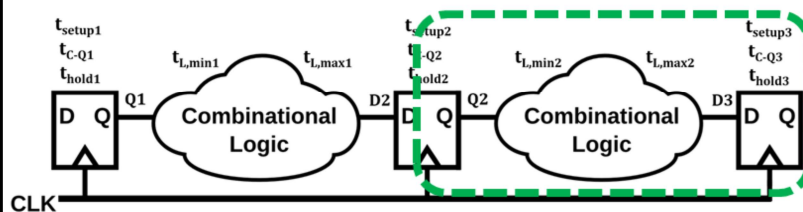
$$T \geq 50ns$$

So, the **minimum clock period T** for FF1 and FF2 must be **at least 50ns** to ensure the circuit operates correctly under the worst-case timing conditions.



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



Path 2: Flipflop 2 to flipflop 3

$$T \geq t_{C-Q2} + t_{Lmax2} + t_{setup3}$$

$$T \geq 7ns + 50ns + 10ns$$

$$T \geq 67ns$$

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

Again, to determine the minimum clock period for two connected flip-flops, consider the maximum value of  $t_{Logic}$ ,  $t_{C-Q}$  of the second flip-flop and the  $t_{setup}$  of the third flip-flop.

Here,  $t_{C-Q2} = 7 \text{ ns}$ ,  $t_{Lmax2} = 50 \text{ ns}$ , and  $t_{setup3} = 10 \text{ ns}$ .

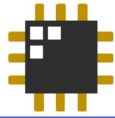
Applying the values to our equation:

$$T \geq t_{C-Q} + t_{Lmax} + t_{setup}$$

$$T \geq 7ns + 50ns + 10ns$$

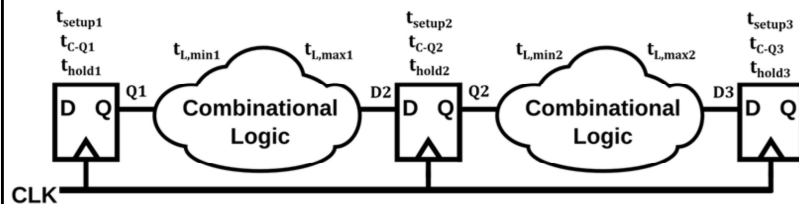
$$T \geq 67ns$$

So, the **minimum clock period T** for FF2 to FF3 must be **at least 67ns** to ensure the circuit operates correctly under the worst-case timing conditions.



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



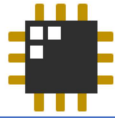
Path 1: Flipflop 1 to flipflop 2  $T \geq 50\text{ns}$

Path 2: Flipflop 2 to flipflop 3  $T \geq 67\text{ns}$

We should pick the worst-case among all the paths (clock period is constant for the system)

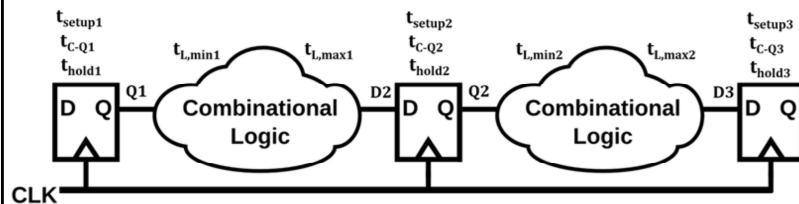
|                     |            |
|---------------------|------------|
| $t_{\text{Logic1}}$ | 10 – 40 ns |
| $t_{\text{Logic2}}$ | 20 – 50 ns |
| $t_{\text{Lmin1}}$  | X          |
| $t_{\text{Lmin2}}$  | Y          |
| $t_{\text{setup1}}$ | 3 ns       |
| $t_{\text{C-Q1}}$   | 4 ns       |
| $t_{\text{hold1}}$  | 5 ns       |
| $t_{\text{setup2}}$ | 6 ns       |
| $t_{\text{C-Q2}}$   | 7 ns       |
| $t_{\text{hold2}}$  | 8 ns       |
| $t_{\text{setup3}}$ | 10 ns      |
| $t_{\text{C-Q3}}$   | 11 ns      |
| $t_{\text{hold3}}$  | 12 ns      |

In a system with multiple flip-flops, different paths might have different delays. The **worst-case path** is the one with the **longest delay**, and this is what ultimately determines the minimum clock period.



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



Path 1: Flipflop 1 to flipflop 2  $T \geq 50\text{ns}$

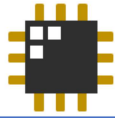
Path 2: Flipflop 2 to flipflop 3  $T \geq 67\text{ns}$

We should pick the worst-case among all the paths (clock period is constant for the system)

|                     |            |
|---------------------|------------|
| $t_{\text{Logic1}}$ | 10 – 40 ns |
| $t_{\text{Logic2}}$ | 20 – 50 ns |
| $t_{\text{Lmin1}}$  | X          |
| $t_{\text{Lmin2}}$  | Y          |
| $t_{\text{setup1}}$ | 3 ns       |
| $t_{\text{C-Q1}}$   | 4 ns       |
| $t_{\text{hold1}}$  | 5 ns       |
| $t_{\text{setup2}}$ | 6 ns       |
| $t_{\text{C-Q2}}$   | 7 ns       |
| $t_{\text{hold2}}$  | 8 ns       |
| $t_{\text{setup3}}$ | 10 ns      |
| $t_{\text{C-Q3}}$   | 11 ns      |
| $t_{\text{hold3}}$  | 12 ns      |

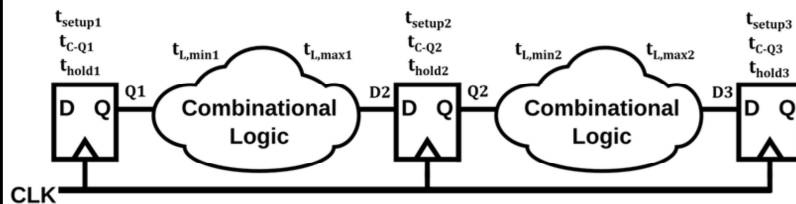
Hence, the minimum clock period must be **67 ns**.





## Practice #2 Part 2

- Given the circuit below and the following timing parameters



$t_{Lmin}$  calculation:

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

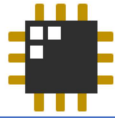
What values of  $t_{C-Q}$  and  $t_{hold}$  should we follow?

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

Now, moving on to the calculation of  $t_{Lmin}$ , we use the inequality:

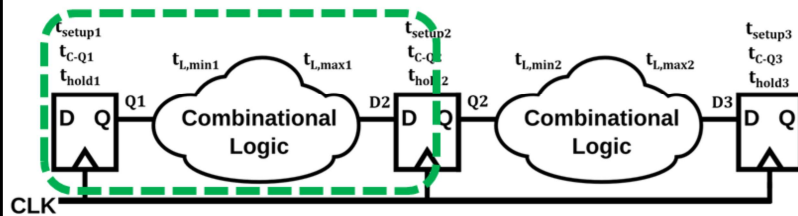
$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

This ensures that the data remains stable long enough after the clock edge for the receiving flip-flop to capture it correctly.



## Practice #2 Part 2

- Given the circuit below and the following timing parameters



Path 1: Flipflop 1 to flipflop 2

$$t_{hold2} \leq t_{C-Q1} + t_{Lmin1}$$

$$8ns \leq 4ns + t_{Lmin1}$$

$$t_{Lmin1} \geq 4ns$$

|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | X          |
| $t_{Lmin2}$  | Y          |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

To determine the  $t_{Lmin}$  for two connected flip-flops, consider the  $t_{C-Q}$  of the first flip-flop and the  $t_{hold}$  of the second flip-flop.

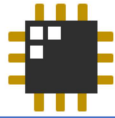
Here, we'll use  $t_{C-Q1} = 4 ns$  and  $t_{hold2} = 8 ns$ .

Using the inequality:

$$8ns \leq 4ns + t_{Lmin1}$$

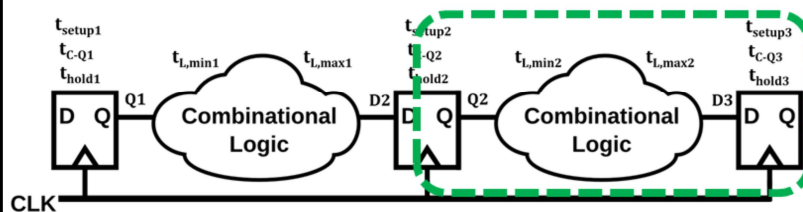
$$t_{Lmin1} \geq 4ns$$

So, the **minimum propagation delay** ( $t_{Lmin}$ ) for FF1 to FF2 is **at least 4ns** to satisfy the hold time requirement.



## Practice #2 Part 2

- Given the circuit below and the following timing parameters



Path 2: Flipflop 2 to flipflop 3

$$t_{\text{hold}3} \leq t_{\text{C-Q}2} + t_{\text{Lmin}2}$$

$$12\text{ns} \leq 7\text{ns} + t_{\text{Lmin}2}$$

$$t_{\text{Lmin}2} \geq 5\text{ns}$$

|                     |            |
|---------------------|------------|
| $t_{\text{Logic}1}$ | 10 – 40 ns |
| $t_{\text{Logic}2}$ | 20 – 50 ns |
| $t_{\text{Lmin}1}$  | X          |
| $t_{\text{Lmin}2}$  | Y          |
| $t_{\text{setup}1}$ | 3 ns       |
| $t_{\text{C-Q}1}$   | 4 ns       |
| $t_{\text{hold}1}$  | 5 ns       |
| $t_{\text{setup}2}$ | 6 ns       |
| $t_{\text{C-Q}2}$   | 7 ns       |
| $t_{\text{hold}2}$  | 8 ns       |
| $t_{\text{setup}3}$ | 10 ns      |
| $t_{\text{C-Q}3}$   | 11 ns      |
| $t_{\text{hold}3}$  | 12 ns      |

To determine the  $t_{\text{Lmin}}$  for two connected flip-flops, consider the  $t_{\text{C-Q}}$  of the second flip-flop and the  $t_{\text{hold}}$  of the third flip-flop.

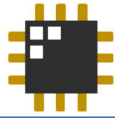
Here, we'll use  $t_{\text{C-Q}2} = 7\text{ ns}$  and  $t_{\text{hold}3} = 12\text{ ns}$ .

Using the inequality:

$$12\text{ns} \leq 7\text{ns} + t_{\text{Lmin}2}$$

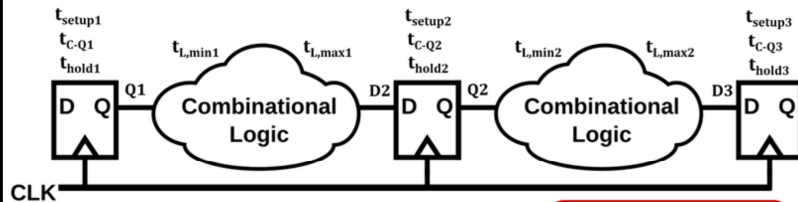
$$t_{\text{Lmin}2} \geq 5\text{ns}$$

So, the **minimum propagation delay** ( $t_{\text{Lmin}}$ ) for FF2 to FF3 is **at least 5ns** to satisfy the hold time requirement.



## Practice #2 Part 1

- Given the circuit below and the following timing parameters



Path 1: Flipflop 1 to flipflop 2

$$t_{Lmin1} \geq 4ns$$

Path 2: Flipflop 2 to flipflop 3

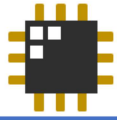
$$t_{Lmin2} \geq 5ns$$

These two values would correspond to different logic blocks and are independent of one another.

**Both must be satisfied to guarantee correct operation.**

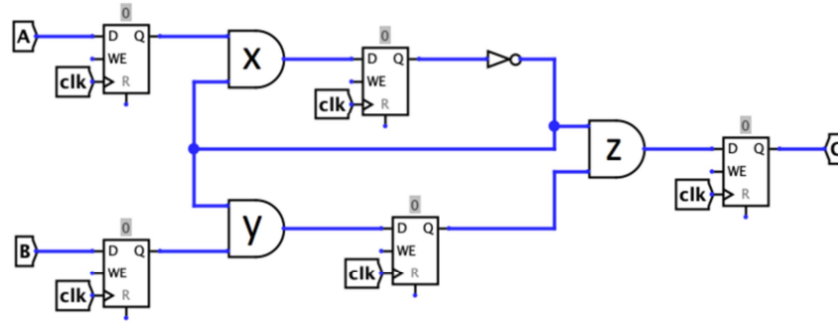
|              |            |
|--------------|------------|
| $t_{Logic1}$ | 10 – 40 ns |
| $t_{Logic2}$ | 20 – 50 ns |
| $t_{Lmin1}$  | 4ns        |
| $t_{Lmin2}$  | 5ns        |
| $t_{setup1}$ | 3 ns       |
| $t_{C-Q1}$   | 4 ns       |
| $t_{hold1}$  | 5 ns       |
| $t_{setup2}$ | 6 ns       |
| $t_{C-Q2}$   | 7 ns       |
| $t_{hold2}$  | 8 ns       |
| $t_{setup3}$ | 10 ns      |
| $t_{C-Q3}$   | 11 ns      |
| $t_{hold3}$  | 12 ns      |

The two minimum propagation delays ( $t_{Lmin}$ ) correspond to different logic blocks and are independent. However, both must be satisfied to ensure correct operation of the system. Each delay affects different paths, and if either is not met, the circuit could behave incorrectly. Therefore, all timing constraints must be respected for proper synchronization and reliable operation.



## Practice #3

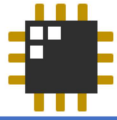
- Given the circuit below and the following assumptions:



- You can safely ignore inputs A and B, and output C for this problem and only consider flipflop-to-flipflop paths.
- The delays of the AND gates (x, y, z) are identical and non-zero, and are equal to the delay of the NOT gate.
- All flipflops are identical to each other (i.e. they have the same clock-to-q delays, setup times, and hold times) and are connected to the same clock input.

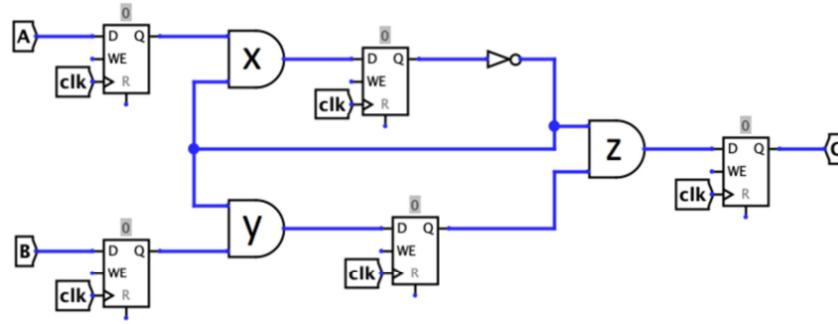
45

Now, let's move on to the final example for this lecture.



## Practice #3 Part 1

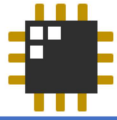
- Given the circuit below and the following assumptions:



- How many timing paths exist in this circuit? (all paths are bounded by flipflops)

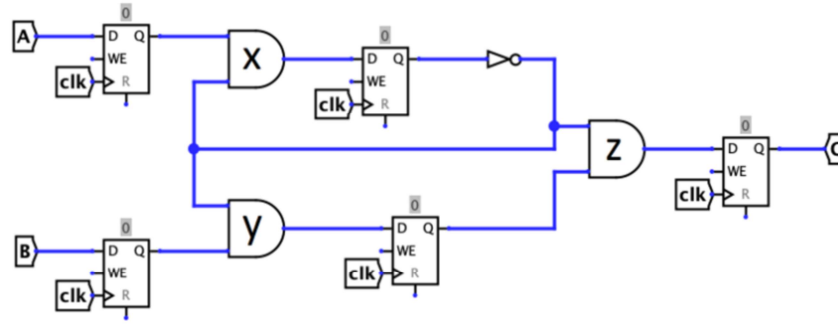
46

Remember, valid paths start from one flipflop output and end at the flipflop input. The starting and ending points of this path can be the same flipflop itself.



## Practice #3 Part 1

- Given the circuit below and the following assumptions:



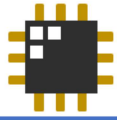
- How many timing paths exist in this circuit? (all paths are bounded by flipflops)

**6 paths!**

Practice: Identify all the paths in the circuit!

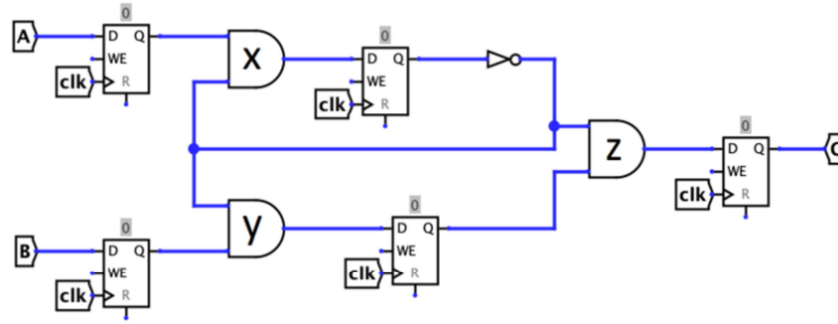
There are six timing paths in this circuit, each with distinct starting and ending points:

1. The path between the two flip-flops connected by the AND gate labeled **X**.
2. The path between the two flip-flops connected by the AND gate labeled **Y**.
3. The path starting from the **NOT gate** and ending at the output of **AND gate Y**.
4. The path starting from the **NOT gate** and ending at the output of **AND gate X**.
5. The path starting from the **NOT gate** and ending at the output of **AND gate Z**.
6. The path between the two flip-flops connected by the AND gate labeled **Z**.



## Practice #3 Part 2

- Given the circuit below and the following assumptions:

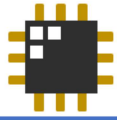


- Assuming all logic gates have a delay of 1ns.
  - What is the shortest logic delay,  $t_{Lmin}$ ?
  - What is the longest logic delay,  $t_{Lmax}$ ?

48

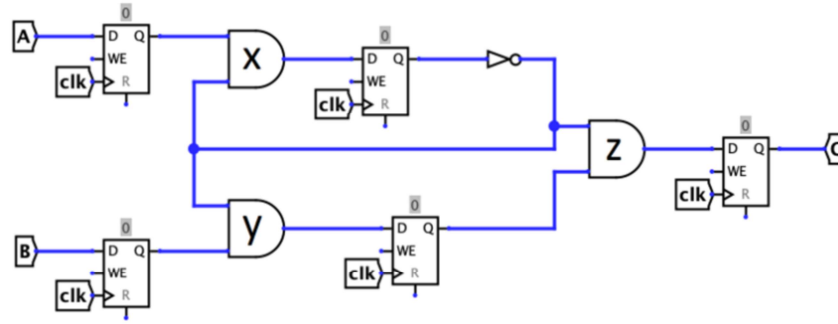
The shortest logic delay occurs along the path that passes through only an AND gate, while the longest logic delay involves both a NOT gate and an AND gate in the path.





## Practice #3 Part 2

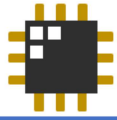
- Given the circuit below and the following assumptions:



- Assuming all logic gates have a delay of 1ns.
  - What is the shortest logic delay,  $t_{Lmin}$ ? **1ns**
  - What is the longest logic delay,  $t_{Lmax}$ ? **2ns**

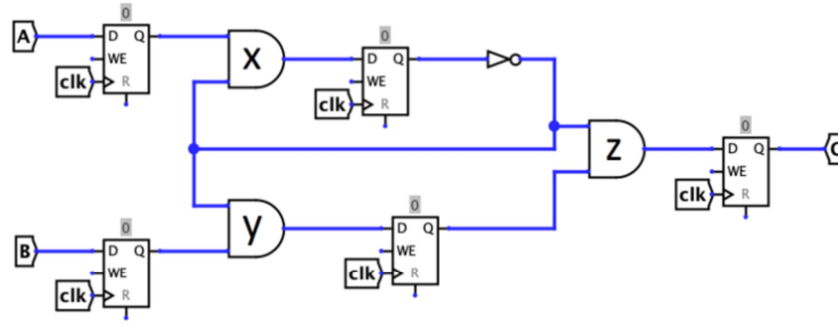
49

Given that all logic gates have a delay of 1ns,  $t_{Lmin} = 1ns$  and  $t_{Lmax} = 2ns$ .



## Practice #3 Part 3

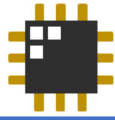
- Given the circuit below and the following assumptions:



- Assuming all logic gates have a delay of 1ns,  $t_{C-Q} = 2\text{ns}$ ,  $t_{\text{setup}} = 3\text{ns}$ ,  $t_{\text{hold}} = 2\text{ns}$ 
  - What is the minimum clock period  $T$  to guarantee correct operation?

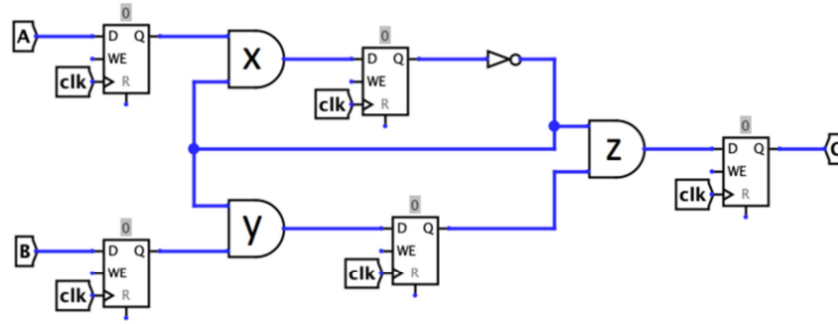
50

Now, let's solve for the minimum clock period  $T$ .



## Practice #3 Part 3

- Given the circuit below and the following assumptions:



- Assuming all logic gates have a delay of 1ns,  $t_{C-Q} = 2\text{ns}$ ,  $t_{\text{setup}} = 3\text{ns}$ ,  $t_{\text{hold}} = 2\text{ns}$ 
  - What is the minimum clock period T to guarantee correct operation?

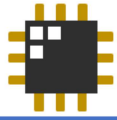
$$T \geq t_{C-Q} + t_{L_{\text{max}}} + t_{\text{setup}} \quad T \geq 2\text{ns} + 2\text{ns} + 3\text{ns}$$

$$T \geq 7\text{ns}$$

Again, remember the equation to use for solving the minimum clock period is:

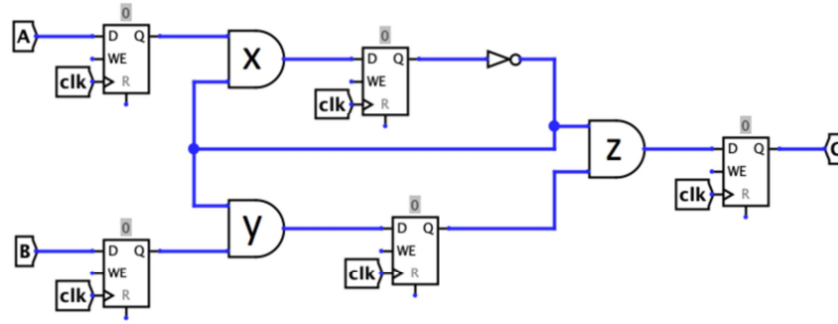
$$T \geq t_{C-Q} + t_{L_{\text{max}}} + t_{\text{setup}}$$

By substituting the given values into the equation, we arrive at the result:  $T \geq 7\text{ns}$ .



## Practice #3 Part 4

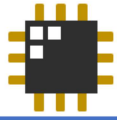
- Given the circuit below and the following assumptions:



- Assuming all logic gates have a delay of 1ns,  $t_{C-Q} = 2\text{ns}$ ,  $t_{\text{setup}} = 3\text{ns}$ ,  $t_{\text{hold}} = 2\text{ns}$ 
  - Is the hold time requirement satisfied?

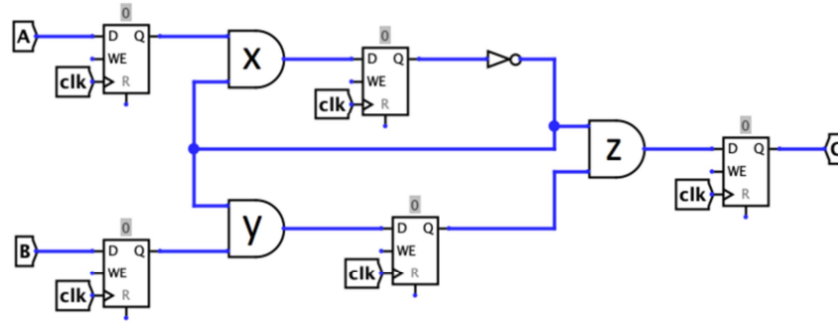
52

Lastly, let's check if the hold time requirement is satisfied.



## Practice #3 Part 4

- Given the circuit below and the following assumptions:



- Assuming all logic gates have a delay of 1ns,  $t_{C-Q} = 2ns$ ,  $t_{setup} = 3ns$ ,  $t_{hold} = 2ns$ 
  - Is the hold time requirement satisfied?

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

$$2ns \leq 2ns + 1ns$$

**Hold time requirement is satisfied!**  
**This circuit will work correctly** 53

Using the equation and substituting the given values,

$$t_{hold} \leq t_{C-Q} + t_{Lmin}$$

$$2ns \leq 2ns + 1ns$$

Since the inequality holds, the **hold time requirement is satisfied**, and the circuit will operate correctly.